

Beginner-Friendly Assignment & Project Guide

Core tasks for everyone + optional Expert Challenges (Day 1 - Day 10)

Contents

Start Here	3
Mini Glossary (plain English)	5
Day 1 - NumPy + Pandas	6
Day 2 - Linear Regression From Scratch	8
Day 3 - Overfitting, Regularization & Cross-Validation	10
Day 4 - Metrics, Trees & Boosting (Fraud Detection)	12
Day 5 - Mini Project 1: Telco Customer Churn	14
Day 6 - Neural Network From Scratch	16
Day 7 - PyTorch Basics	18
Day 8 - Mini Project 2: MNIST, MLP vs CNN	20
Day 9 - Self-Attention	22
Final Project - Day 10 Capstone	24
Progress Tracker	27
Solutions Appendix	28
Solution - Day 1	29
Solution - Day 2	31
Solution - Day 3	33
Solution - Day 4	35
Solution - Day 5	37
Solution - Day 6	39
Solution - Day 7 (PyTorch, not run while building this guide)	41
Solution - Day 8 (PyTorch, not run while building this guide)	43
Solution - Day 9 (PyTorch, not run while building this guide)	45

Solution - Day 10 (Capstone)

47

Start Here

How this guide works

Every day has two levels. **Do Level 1 first. Only do Level 2 if you have time and energy.**

Level	Name	Who it is for	What it gives you
Level 1	Core (Must do)	Everyone	You finish the assignment and understand it
Level 2	Expert Challenge (Optional)	When Level 1 feels easy	Skills that make you stand out

You do NOT need Level 2 to complete the course. A finished Level 1 beats an unfinished Level 2 every time.

Each day is laid out the same way

1. **Goal in plain words** - what you are doing and why
2. **Words you need** - short definitions, no jargon
3. **Level 1 - Must do** - small numbered steps
4. **You are done when...** - a clear finish line
5. **If you get stuck** - the most common problem and its fix
6. **Hints** - three tiers, open one at a time (before peeking at solutions)
7. **Fill-in README** - sentences with blanks to complete
8. **Level 2 - Expert Challenges** - optional extras
9. **Solutions** - complete code for every day in the appendix at the end

How to use Hints and Solutions

Stage	What to do
Try first	Spend at least 20 minutes on the Level 1 steps alone
Hint 1	A gentle nudge: tells you <i>where</i> to look
Hint 2	More specific: tells you <i>which function or idea</i>
Hint 3	Almost the answer: shows the key line
Solution	Full working code in the Solutions Appendix at the end. Read it, close it, then type your own version

Never copy a solution. If you looked at it, close it and rewrite it from memory. That is where the learning happens.

The solutions were run on stand-in data where the real dataset needs a download (Titanic, fraud, churn), so your exact numbers will differ a little. The PyTorch and MNIST code (Days 7, 8, 9) follows your notes but could not be run in the environment that built this guide, so if a line errors, check the shapes first and compare with your Day notes.

The golden rules for beginners

- **Type the code yourself.** Do not copy-paste. Typing teaches you.
- **Run small pieces.** Run one cell at a time and look at the output.
- **Print shapes and heads.** `print(x.shape)` and `df.head()` solve half of all bugs.
- **Stuck for 15 minutes? Take a break, then read the error message from the bottom line up.** The last line says what went wrong.
- **One day = one folder = one README.** Small and finished is the goal.

Setup checklist (do once)

- ☐ Install Python 3.10 or newer
- ☐ Run: `pip install numpy pandas matplotlib scikit-learn jupyter torch seaborn`
- ☐ Open a notebook: `jupyter notebook` (or use VS Code)
- ☐ Create a GitHub account and an empty repo called `ml-dl-fundamentals`
- ☐ Create one folder per day, e.g. `Day-01-NumPy-Pandas`

Git in 4 commands (for pushing your work)

```
git add .
git commit -m "Finish Day 1"
git push
```

Do this at the end of every day. Your GitHub becomes your portfolio.

The 3 passes (a simple work rhythm)

Pass	What you do
1. Make it work	Run the code, get the right answer
2. Understand it	Explain each line in your own words
3. Tidy it	Add titles to plots, write the README, Restart and Run All

Tidy checklist (use for every day)

- ☐ Notebook has a title and short headings
- ☐ **Kernel -> Restart and Run All** works with no red errors
- ☐ Every plot has a title, axis labels and (if needed) a legend
- ☐ README has a results table and at least 3 sentences of insight
- ☐ Big data files are NOT uploaded to GitHub

Mini Glossary (plain English)

Come back to this page whenever a word confuses you.

Word	Simple meaning
Array	A grid of numbers (NumPy's basic object)
Shape	The size of an array, like (3, 4) = 3 rows, 4 columns
Vectorization	Doing math on a whole array at once instead of a for loop
Broadcasting	NumPy stretches a small array to match a bigger one
DataFrame	A table (rows and columns) in Pandas
Missing values (NaN)	Empty cells in your data
Encoding	Turning words (like "male") into numbers so a model can use them
One-hot encoding	One new 0/1 column for each category
Feature	An input column (like age)
Target / label	The thing you want to predict (like Survived)
Train set	Data the model learns from
Validation set	Data used to compare and tune models
Test set	Data used once at the very end for the final score
Loss	A score of how wrong the model is (lower is better)
Gradient descent	Repeatedly nudging the weights to make the loss smaller
Learning rate	How big each nudge is
Epoch	One full pass through the training data
Overfitting	Memorizes training data, fails on new data
Underfitting	Too simple, bad on everything
Regularization	A penalty that keeps the model from getting too wild (Ridge, Lasso)
Cross-validation (CV)	Testing on several different splits to get a more trustworthy score
Data leakage	Accidentally letting test information into training (gives fake good scores)
Pipeline	A chain of steps (scale, encode, model) that is applied safely and in order
Stratify	Keep the same class ratio in train and test
Imbalanced data	One class is rare (like fraud, 0.17%)
Precision	Of the ones I flagged, how many were truly positive?
Recall	Of all the truly positive ones, how many did I catch?
F1	One number that balances precision and recall
ROC-AUC	How well the model separates the two classes (1.0 perfect, 0.5 random)
Confusion matrix	A 2x2 table of right and wrong predictions
Backpropagation	Working out how each weight affected the loss, going backwards
Tensor	PyTorch's version of an array
Dropout	Randomly switching off neurons during training to reduce overfitting
Early stopping	Stop training when validation score stops improving
CNN	A network for images that scans small patches
Attention	Each word looks at other words and decides which matter most
Q, K, V	Query (what I seek), Key (what I offer), Value (what I give)

Day 1 - NumPy + Pandas

Goal in plain words

Learn to do fast math on many numbers at once (NumPy) and to clean a messy table (Pandas). Every later day uses these tools.

Words you need

Array, shape, vectorization, DataFrame, NaN, encoding (see the glossary). **Cosine similarity** = a score from -1 to 1 saying how similar two lists of numbers are (1 = same direction).

Level 1 - Must do

Assignment A: Cosine similarity

1. Write a function `cosine(a, b)` for two vectors: `np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))`
2. Test by hand: `a=[1,0]`, `b=[1,1]` should give about **0.707**
3. Make two random matrices: `A = np.random.rand(4, 8)` and `B = np.random.rand(5, 8)`
4. Normalize each row (make its length 1): `A_n = A / np.linalg.norm(A, axis=1, keepdims=True)`, same for B
5. Compute all similarities at once: `A_n @ B_n.T`. The result shape should be (4, 5)

Assignment B: Titanic cleaning

1. Load the file and look: `df.head()`, `df.shape`, `df.isna().sum()`
2. Drop columns you don't need: Cabin, Name, Ticket, PassengerId
3. Fill missing Age with the median and missing Embarked with the most common value
4. Turn Sex into numbers (male=0, female=1)
5. One-hot encode Embarked with `pd.get_dummies(..., drop_first=True)`
6. Check there are zero missing values, then save with `df.to_csv("titanic_clean.csv", index=False)`
7. Answer with GroupBy: who survived more, women or men? Which class?

Hints

A1. Cosine of two vectors

- *Hint 1:* The top of the formula is a dot product; the bottom is two lengths multiplied.
- *Hint 2:* NumPy has `np.dot(a, b)` and `np.linalg.norm(a)`.
- *Hint 3:* `np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))`

A2. Whole matrix at once

- *Hint 1:* If every row has length 1, cosine is just a dot product.
- *Hint 2:* Divide each row by its own length. `axis=1` means "along each row"; `keepdims=True` keeps the shape (n, 1) so the division works.
- *Hint 3:* `A_n @ B_n.T` gives shape (n, m).

B. Titanic

- *Hint 1:* Look at `df.isna().sum()` first. Only columns with a number above 0 need fixing.
- *Hint 2:* Numbers use `fillna(median)`; categories use `fillna(mode()[0])`.
- *Hint 3:* `df.groupby("Sex")["Survived"].mean()` answers "who survived more?"

You are done when...

- ☐ Your (4, 5) similarity table appears with no loops
- ☐ `titanic_clean.csv` has no missing values and only numbers
- ☐ README lists 3 things you found in the data

If you get stuck

Problem	Fix
"shapes not aligned" KeyError: 'Age' Model complains about text	Print <code>A.shape</code> and <code>B.shape</code> ; you probably forgot <code>.T</code> Print <code>df.columns</code> ; check spelling and capital letters A column is still words; encode it

Fill-in README

I cleaned the Titanic data by dropping ____ columns and filling missing ____ with the _____. I found that women survived at about ____% and men at about ____%. First class passengers survived ____ than third class. My cosine similarity matrix has shape _____.

Level 2 - Expert Challenges (optional)

1. **Loop vs vectorized race:** write the slow double-loop version, check `np.allclose` with your fast one, and time both with `%timeit`. Show the speed-up.
2. **Zero-vector safety:** make your function handle a row of all zeros without crashing.
3. **Heatmap:** plot the similarity matrix with `plt.imshow` and a colorbar.
4. **Chart the story:** make 3 plots (survival by sex, by class, age histogram).
5. **Safety net:** add `assert df.isna().sum().sum() == 0` so the notebook fails loudly if cleaning is incomplete.

Day 2 - Linear Regression From Scratch

Goal in plain words

Teach a computer to draw the best straight line through points, by making small corrections again and again. That process is **gradient descent**.

Words you need

Loss (MSE), learning rate, epoch, gradient descent (see the glossary). **Mountain picture:** you stand on a hill in fog and take small steps downhill. The hill height is the loss; the step size is the learning rate.

Level 1 - Must do

1. Make fake data with the true rule $y = 4 + 3x + \text{noise}$ (`np.random.seed(42)`, 100 points)
2. Start with $w = 0$, $b = 0$, $lr = 0.1$, `epochs = 200`
3. In a loop, do these five things:
 - predict: $y_{\text{pred}} = w * X + b$
 - error: $y_{\text{pred}} - y$
 - loss: mean of error squared
 - gradients: dw , db (formulas are in your notes)
 - update: $w = w - lr * dw$, $b = b - lr * db$
4. Save the loss each epoch in a list
5. Plot the loss curve (it should go down and flatten)
6. Plot the data points and your fitted line
7. Fit `LinearRegression()` from scikit-learn and compare its w and b with yours

Hints

- *Hint 1 (predict):* One line: $w * X + b$.
- *Hint 2 (gradients):* Both gradients start with $(2/n) * \text{np.sum}(\text{error} \dots)$. For dw the error is multiplied by X ; for db it is not.
- *Hint 3 (update):* Move **against** the slope: $w = w - lr * dw$.
- *Loss goes up?* Your learning rate is too big. Divide it by 10.
- *Comparing with sklearn:* `model.coef_[0][0]` is w and `model.intercept_[0]` is b .

You are done when...

- ☐ Your w is near **3** and b is near **4** (with seed 42 you should get about **2.8** and **4.2**, because the noise nudges the answer)
- ☐ Your numbers are close to scikit-learn's
- ☐ Loss curve goes down smoothly

If you get stuck

Problem	Fix
Loss goes UP or becomes nan	Learning rate is too big; try 0.01
Loss barely moves	Learning rate too small, or too few epochs
Shape errors	Print <code>X.shape</code> and <code>y.shape</code> ; both should be <code>(100, 1)</code>

Fill-in README

My gradient descent found $w = \underline{\hspace{1cm}}$ and $b = \underline{\hspace{1cm}}$ (true values 3 and 4). scikit-learn found $w = \underline{\hspace{1cm}}$ and $b = \underline{\hspace{1cm}}$. The loss curve $\underline{\hspace{1cm}}$ (goes down and flattens). When I made the learning rate too large, $\underline{\hspace{1cm}}$.

Level 2 - Expert Challenges (optional)

1. **Learning-rate lab:** run $lr = 0.001, 0.01, 0.1, 0.5, 1.5$ and plot all loss curves together. Label “too slow”, “good”, “explodes”.
2. **Turn it into a function:** `fit(X, y, lr, epochs)` that returns w, b, losses .
3. **Many features:** rewrite using $y_{\text{pred}} = X @ w + b$ for 3 input columns.
4. **Mini-batch version:** update using random batches of 16 instead of all 100 rows.
5. **Check the math:** confirm your gradient formula against a numerical estimate $(L(w+e) - L(w-e)) / (2e)$.

Day 3 - Overfitting, Regularization & Cross-Validation

Goal in plain words

See with your own eyes how a model can be **too simple**, **just right** or **too wild**, and learn the tools that keep it under control.

Words you need

Overfitting, underfitting, train/validation/test, Ridge, Lasso, cross-validation, data leakage. Student picture: overfitting = a student who memorizes past exam answers but fails a new exam.

Level 1 - Must do

1. Make curved data (a cubic curve plus noise; code is in your notes)
2. Split into train and validation (`test_size=0.3`)
3. For degrees 1 to 15: fit a polynomial model and record **train error** and **validation error**
4. Plot both errors against degree
5. Find the degree where validation error is lowest (about 3)
6. At degree 15 compare three models: plain Linear, **Ridge**, **Lasso**. Print each validation error
7. Run **5-fold cross-validation** on: Degree 3 Linear, Degree 15 Linear, Degree 15 Ridge. Print mean and std

Hints

- *Hint 1:* You need a loop over `d in range(1, 16)`. Inside, build one model, fit it, record two errors.
- *Hint 2:* The model is `make_pipeline(PolynomialFeatures(d), StandardScaler(), LinearRegression())`.
- *Hint 3:* `Error = mean_squared_error(y_true, model.predict(X))`. Do it once for train and once for validation.
- *Finding the best degree:* `np.argmin(val_err) + 1` (add 1 because Python starts at 0).
- *Cross-validation:* `cross_val_score(model, X, y, cv=5, scoring="neg_mean_squared_error")` returns **negative** errors, so put a minus sign in front.
- *Don't panic:* Degree-15 plain Linear giving a gigantic CV error is **normal**. That is overfitting at its worst.

You are done when...

- ☐ Your plot shows train error falling and validation error making a "U" shape
- ☐ You can say which model is best and why
- ☐ You have a results table in the README

If you get stuck

Problem	Fix
Lasso warning about convergence	Add <code>max_iter=50000</code>
Errors are huge for high degrees	Normal for plain Linear at degree 15; that IS overfitting
Not sure what to conclude	Compare validation errors; smallest wins

Fill-in README

Training error kept going ____ as degree increased, but validation error went down until degree ____ and then went _____. This shows _____. At degree 15, ____ (Ridge/Lasso) performed ____ than plain Linear because regularization _____. The best model in cross-validation was ____ with mean error _____.

Level 2 - Expert Challenges (optional)

1. **Shade the zones:** mark “Underfit / Good / Overfit” on your plot with a dashed line at the best degree.
2. **Show the curves:** draw the fitted curve for degrees 1, 3 and 15 over the data points.
3. **Alpha experiment:** try $\alpha = 0.01, 1, 100$ for Ridge and describe what happens.
4. **Add Lasso to CV** and check how many coefficients became exactly 0.
5. **Leakage demo:** scale using ALL data vs train only, and show the difference in score.
6. **Learning curve:** plot error against training set size (20, 40, 60...).

Day 4 - Metrics, Trees & Boosting (Fraud Detection)

Goal in plain words

Learn why “99% accuracy” can be a useless score, and how to judge a model when one class is very rare (fraud).

Words you need

Imbalanced data, precision, recall, F1, ROC-AUC, confusion matrix, stratify. Fire alarm picture: high recall = the alarm catches almost every fire. High precision = when it rings, there really is a fire.

Level 1 - Must do

1. Load `creditcard.csv`. Print how many rows are fraud (`Class = 1`); it is about 0.17%
2. Split with `stratify=y` so both sets have fraud cases
3. Scale Amount and Time (learn the scaling from the train set only)
4. Train three models: **Logistic Regression, Random Forest, Gradient Boosting** (use `class_weight="balanced"` when available)
5. For each model print the confusion matrix and calculate precision, recall, F1, ROC-AUC
6. Put the results into one small table (a DataFrame)
7. Write which model you would give a bank, and why

Hints

- *Hint 1:* Fraud is very rare, so the split must keep that ratio: add `stratify=y`.
- *Hint 2:* Fit the scaler on train only: `scaler.fit(X_train[...])`, then transform train and test.
- *Hint 3:* Loop over a dictionary of models. For each: `fit, predict, predict_proba(X_test)[: , 1]`, then the four scores.
- *Which metric?* For “did we catch the fraud?” look at **recall**. For “are our alarms real?” look at **precision**.
- *Too slow?* Take a sample, e.g. `df.sample(50000, random_state=1)`, but keep all fraud rows.

You are done when...

- ☐ You have a table with 3 models and 4 scores each
- ☐ You can explain why accuracy alone would be misleading
- ☐ You wrote a 3-line recommendation

If you get stuck

Problem	Fix
Boosting takes forever	Use a smaller sample (e.g. 50,000 rows) or <code>HistGradientBoostingClassifier</code>
AUC looks wrong Confusion matrix confusing	Use <code>predict_proba(...)[: , 1]</code> , not <code>predict</code> sklearn layout is <code>[TN, FP], [FN, TP]</code>

Fill-in README

Only ____% of transactions are fraud, so a model that always says “not fraud” gets ____% accuracy but ____% recall. My best recall was ____ (model: ____). My best precision was

____ (model: ____). For a bank I would choose ____ because missing fraud costs ____ while a false alarm only ____.

Level 2 - Expert Challenges (optional)

1. **Dumb baseline:** show “always predict not-fraud” first (great accuracy, zero recall).
2. **Curves:** draw ROC curves and **precision-recall curves** for all models on one chart.
3. **Threshold tuning:** change the cut-off from 0.5 to 0.3 or 0.7 and show how precision and recall trade off.
4. **SMOTE:** oversample fraud in the TRAIN set only, then compare.
5. **Feature importance:** show the top 10 features for Random Forest.
6. **Cost thinking:** assume missed fraud costs 100 and a false alarm costs 1; pick the model with the lowest total cost.

Day 5 - Mini Project 1: Telco Customer Churn

Goal in plain words

Build your first complete machine learning project: predict which phone customers will leave (churn), and explain why.

Words you need

Pipeline, ColumnTransformer, cross-validation, feature importance, churn. Assembly line picture: a Pipeline is a conveyor belt: clean, scale, encode, predict, always in the same order, always learning only from training data.

Level 1 - Must do

1. Load the CSV; print shape, info() and how many customers churn (about 27%)
2. Clean:
 - TotalCharges -> numbers with `pd.to_numeric(..., errors="coerce")`, then fill blanks with the median
 - drop customerID
 - change Churn from Yes/No to 1/0
3. Split into train and test (stratify=y, 80/20)
4. Build preprocessing: scale the number columns, one-hot encode the text columns (ColumnTransformer)
5. Train two models: **Logistic Regression** and **Random Forest** (each inside a Pipeline)
6. Check each with 5-fold cross-validation (look at recall and ROC-AUC)
7. Pick the better model, then evaluate on the test set **once**
8. Show the top features that drive churn
9. Write the README

Hints

- *Hint 1:* TotalCharges is text because some cells are a blank space. Convert with `pd.to_numeric(..., errors="coerce")`.
- *Hint 2:* Split the columns: numbers are tenure, MonthlyCharges, TotalCharges; everything else is categories.
- *Hint 3:* `ColumnTransformer([("num", StandardScaler(), num_cols), ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)])`, then `Pipeline([("pre", pre), ("clf", model)])`.
- *Choosing the model:* Compare CV **recall and ROC-AUC**. Don't look at the test set until the very end.
- *Feature names:* `pipeline.named_steps["pre"].get_feature_names_out()`.

You are done when...

- ☐ You have a table comparing the models
- ☐ Your final ROC-AUC is roughly **0.80 to 0.85**
- ☐ README explains in plain words **why customers leave**

If you get stuck

Problem	Fix
TotalCharges is text	Convert with <code>to_numeric(errors="coerce")</code>
Scores are suspiciously perfect	You may have leaked the target; check Churn isn't in X
Feature names look ugly	Use <code>get_feature_names_out()</code> from the preprocessor

Fill-in README

Problem: A telecom wants to predict which customers will leave so it can offer them deals.

Data: ____ customers, ____% churn. **Method:** Pipeline with scaling and one-hot encoding, models: ____, checked with 5-fold CV. **Results:** best model ____ with ROC-AUC ____ and recall ____ on the test set. **Insights:** customers with ____ contracts, ____ tenure and ____ internet are most likely to leave. **Recommendation:** ____.

Level 2 - Expert Challenges (optional)

1. **EDA first:** make 3 plots (churn by contract type, tenure histogram by churn, monthly charges by churn).
2. **Add Gradient Boosting** and report mean +/- std from CV.
3. **Hyperparameter search:** use GridSearchCV on Random Forest (max_depth, n_estimators).
4. **Save the model:** `joblib.dump(pipeline, "churn.pkl")` and write a `predict_one(customer)` example.
5. **Business value:** estimate how much money is saved if you contact the top 20% riskiest customers.
6. **Unsupervised bonus:** run k-Means on customers and describe the groups.

Day 6 - Neural Network From Scratch

Goal in plain words

Build a tiny neural network using only NumPy so nothing is hidden. After this, PyTorch will feel like a shortcut, not magic.

Words you need

Neuron, activation (ReLU, sigmoid), forward pass, loss, backpropagation, hidden layer.

Idea: forward pass = make a guess. Loss = how wrong. Backprop = who's to blame. Update = fix the weights a little.

Level 1 - Must do

1. Make the moons dataset (1,000 points, noise=0.2), split 80/20
2. Write three helper functions: relu, sigmoid, and the loss (bce, with a tiny eps so $\log(0)$ never happens)
3. Create the weights: small random numbers; biases start at zero
4. Loop for many epochs:
 - **forward:** layer 1, ReLU, layer 2, sigmoid
 - **loss**
 - **backward:** compute $dw2$, $db2$, $dw1$, $db1$ (copy the formulas from your notes and understand them)
 - **update:** subtract $lr * \text{gradient}$
5. Print accuracy on train and test
6. Plot the loss curve

Hints

- *Hint 1 (shapes):* Write the shape of every array on paper first: X (n , 2), $W1$ (2, 16), hidden (n , 16), $W2$ (16, 1), output (n , 1).
- *Hint 2 (backward):* Start at the end: $dz2 = (a2 - y) / N$. Then walk backwards, one layer at a time.
- *Hint 3 (ReLU backward):* The gradient passes only where the input was positive: $dz1 = da1 * (z1 > 0)$.
- *Golden rule:* A gradient has the **same shape** as the thing it belongs to ($dw1$ is (2, 16)).
- *Loss is nan?* Clip probabilities: `np.clip(p, 1e-9, 1 - 1e-9)` and lower lr .

You are done when...

- ☐ Loss goes down steadily
- ☐ Test accuracy is about **95% or more**
- ☐ You can explain forward and backward in your own words

If you get stuck

Problem	Fix
Loss is nan	Lower the learning rate; add eps inside the log
Accuracy stays 50%	Make sure y has shape (n , 1), not (n ,)
Shape mismatch	A gradient must have the same shape as its weight; print shapes

Fill-in README

My network has ____ inputs, ____ hidden neurons and ____ output. It reached ____% test accuracy. The loss started at ____ and ended at _____. Backpropagation means _____. A network without a hidden layer only reaches about ____% because its boundary can only be a _____.

Level 2 - Expert Challenges (optional)

1. **Decision boundary:** plot the boundary the network learned on the moons.
2. **Experiment table:** hidden size (2, 4, 16, 64), learning rate (0.001 to 1.0), ReLU vs tanh; write what changed.
3. **Gradient check:** compare your analytic gradient with the numerical one; they should agree to about 6 decimals.
4. **Class version:** wrap everything in class `NeuralNet` with `.fit()` and `.predict()`.
5. **Third layer:** add a second hidden layer and generalize your backprop.
6. **Debug journal:** list every bug you hit and how you fixed it.

Day 7 - PyTorch Basics

Goal in plain words

Redo the Day 6 network with PyTorch, which computes gradients for you, and see how the optimizer and learning rate change training.

Words you need

Tensor, autograd, DataLoader, nn.Module, optimizer (SGD, Adam), batch. The 5-line training step (memorize it):

```
optimizer.zero_grad()    # 1. clear old gradients
out = model(x)           # 2. forward
loss = criterion(out, y)  # 3. compute loss
loss.backward()          # 4. compute gradients
optimizer.step()         # 5. update weights
```

Level 1 - Must do

1. Turn moons data into tensors; wrap in TensorDataset and DataLoader (batch size 32)
2. Build the model: `nn.Sequential(nn.Linear(2,16), nn.ReLU(), nn.Linear(16,1))`
3. Use BCEWithLogitsLoss and the Adam optimizer (lr 0.01)
4. Write the training loop with the 5-line step for 60 epochs
5. Add a validation loop (use `model.eval()` and `torch.no_grad()`)
6. Print train loss, validation loss and validation accuracy every 10 epochs
7. Plot both losses

Hints

- *Hint 1:* Data must be `torch.float32` tensors. Labels need shape `(n, 1): .view(-1, 1)`.
- *Hint 2:* BCEWithLogitsLoss already includes the sigmoid, so the model's last layer outputs raw numbers (no sigmoid).
- *Hint 3:* The order inside the loop: `zero_grad()`, `forward`, `loss`, `backward()`, `step()`.
- *Validation:* wrap it in `model.eval()` and with `torch.no_grad():`.
- *Accuracy:* `(torch.sigmoid(out) > 0.5).float() == yb, then .sum()`.

You are done when...

- ☐ Validation accuracy is about **90 to 93%** (this data is noisy, so ~92% is normal)
- ☐ You can write the 5-line step from memory
- ☐ You can explain what `zero_grad()` does

If you get stuck

Problem	Fix
Loss doesn't go down	Did you call <code>zero_grad()</code> , <code>backward()</code> and <code>step()</code> ?
Shape error in loss	Make labels <code>.view(-1, 1)</code> and <code>float</code>
Validation looks random	Did you switch to <code>model.eval()</code> ?

Fill-in README

PyTorch handles ____ automatically with `loss.backward()`. My model reached ____% validation accuracy (about 92% is normal). `zero_grad()` is needed because ____.

`model.eval()` is used during validation because _____. Compared to my NumPy network from Day 6 the result was _____.

Level 2 - Expert Challenges (optional)

1. **Optimizer lab:** run SGD (0.001, 0.01, 0.1) and Adam (0.0001, 0.001, 0.01) and fill this table.

Optimizer	LR	Final train loss	Final val loss	Val acc
-----------	----	------------------	----------------	---------

2. **Plot all 6 runs** with one colour per optimizer.
3. **Count parameters:** `sum(p.numel() for p in model.parameters())`.
4. **Early stopping:** stop when validation loss hasn't improved for 10 epochs.
5. **Custom class:** rewrite the model as `class Net(nn.Module)`.
6. **GPU:** move model and data to cuda if available.

Day 8 - Mini Project 2: MNIST, MLP vs CNN

Goal in plain words

Teach a computer to read handwritten digits, using two kinds of networks, and find out which one is better.

Words you need

MLP, CNN, dropout, early stopping, batch, epoch, confusion matrix. Picture: an MLP looks at the image as one long list of pixels; a CNN looks at small patches, like scanning with a magnifying glass, so it understands shapes.

Level 1 - Must do

1. Load MNIST with torchvision; normalize the images
2. Split the training data into train (55k) and validation (5k); keep the 10k test set aside
3. Build an **MLP**: flatten, Linear(784,256), ReLU, dropout, Linear(256,10)
4. Train it with the 5-line step; use CrossEntropyLoss
5. Build a small **CNN**: Conv, ReLU, Pool (twice), flatten, Linear
6. Train the CNN the same way
7. Record validation accuracy per epoch for both
8. After training, test both **once** on the test set
9. Fill a table: parameters, val accuracy, test accuracy

Hints

- *Hint 1 (data):* `torchvision.datasets.MNIST(..., transform=transforms.ToTensor())`, then `random_split(train, [55000, 5000])`.
- *Hint 2 (CNN size):* After two Conv (padding=1) + MaxPool(2) blocks, a 28x28 image becomes 7x7. With 32 channels the flatten size is $32 * 7 * 7 = 1568$.
- *Hint 3 (MLP):* Start the model with `nn.Flatten()` so the image (B,1,28,28) becomes (B,784).
- *Loss:* `CrossEntropyLoss` wants raw scores (B, 10) and integer labels (B,). Do **not** add softmax yourself.
- *Test once:* Use the test loader only after training is completely finished.

You are done when...

- ☐ MLP is around **97 to 98%**, CNN around **98.5 to 99%**
- ☐ Table with both models is in the README
- ☐ You can explain in 2 sentences why the CNN wins

If you get stuck

Problem	Fix
"mat1 and mat2 shapes cannot be multiplied"	Wrong flatten size; print the shape after the conv layers
"Expected 4D input"	Images need shape (B, 1, 28, 28)
CrossEntropy complains	Labels must be integers (class numbers), shape (B,)
Very slow	Use a GPU if you have one, or fewer epochs

Fill-in README

Model	Parameters	Val acc	Test acc
MLP	___	___	___
CNN	___	___	___

The CNN performed ___ because it keeps the ___ structure of the image. Dropout helps by ___. The digits most often confused were ___ and ___.

Level 2 - Expert Challenges (optional)

1. **Early stopping** with best-weights saving (load_state_dict).
2. **Confusion matrix** heatmap for the CNN.
3. **Show mistakes:** plot 8 wrongly classified digits with true vs predicted labels.
4. **Visualize filters:** show the first-layer CNN filters.
5. **Data augmentation:** small random rotations; see if accuracy improves.
6. **Add BatchNorm** and compare.
7. **Demo cell:** load a digit you draw yourself and predict it.

Day 9 - Self-Attention

Goal in plain words

Understand the idea behind ChatGPT, Claude and other language models: every word looks at the other words and decides which are important.

Words you need

Attention, Q/K/V, softmax, multi-head, mask, positional encoding. **Library picture:** your **Query** is what you're looking for, each book's **Key** is its label, and the **Value** is the content inside. You compare your query to all labels, then read mostly the best-matching books.

The formula in one line:

```
Attention = softmax( Q times K-transpose / sqrt(d_k) ) times V
```

Level 1 - Must do

1. Write the SelfAttention class with three linear layers for Q, K, V
2. In forward: compute scores, divide by $\sqrt{d_k}$, apply softmax, multiply by V
3. Test with `x = torch.randn(1, 6, 16)` (1 sentence, 6 words, 16 numbers each)
4. Check the output shape is (1, 6, 16) and weights shape is (1, 6, 6)
5. Check that **each row of the weights adds up to 1**
6. Plot the weights as a heatmap with word labels
7. Explain Q, K and V in your own words in the README

Hints

- *Hint 1:* Three `nn.Linear` layers with `bias=False` make Q, K and V from the same input x.
- *Hint 2:* Scores are Q times K-transpose: `Q @ K.transpose(-2, -1)`, then divide by `math.sqrt(d_k)`.
- *Hint 3:* `weights = F.softmax(scores, dim=-1)`, and the output is `weights @ V`.
- *Row-sum check:* `weights[0].sum(dim=-1)` should print all ones.
- *Heatmap looks random?* That is expected with untrained weights. It proves the code runs, not that the model is smart.

You are done when...

- ☐ Shapes and row-sum check pass
- ☐ You have a heatmap image
- ☐ Your README explains Q/K/V without copying the notes

If you get stuck

Problem	Fix
Heatmap looks random	Normal! Untrained weights have no meaning
Shape errors in @	Use <code>K.transpose(-2, -1)</code>
Rows don't sum to 1	Apply softmax on the last dimension (<code>dim=-1</code>)

Fill-in README

Attention lets each word _____. **Q** is ____, **K** is ____, **V** is _____. We divide by $\sqrt{d_k}$ because _____. Each row of the attention matrix sums to _____ because of _____. Multi-head attention helps because _____.

Level 2 - Expert Challenges (optional)

1. **Causal mask:** block future words and confirm the weight matrix is lower-triangular (this is how GPT-style models work).
2. **Real model:** load pretrained BERT and plot real attention for *“The animal didn’t cross the street because it was too tired”*. Write 4 to 5 lines on what “it” attends to.
3. **Multi-head attention:** implement it with 4 heads and concatenate.
4. **Full Transformer block:** attention, add and norm, feed-forward, add and norm.
5. **Diagram:** draw the block by hand with every shape labelled.
6. **Verify by hand:** repeat the small numeric example from your notes with a calculator.

Final Project - Day 10 Capstone

Goal in plain words

Rebuild one model **by yourself** from a blank file, explain it clearly, and publish it. This is your portfolio piece.

Choose your path

Path	Project	Difficulty	Good if...
Path 1 (recommended)	MNIST CNN in PyTorch (Option C)	Medium	You want a solid, safe capstone
Path 2 (easier)	Linear or logistic regression from scratch (Option A)	Easy	You are short on time
Path 3 (medium)	2-layer network in NumPy (Option B)	Medium	You loved Day 6
Path 4 (expert, optional)	Self-attention plus tiny text classifier (Option D)	Hard	You want GenAI skills

If unsure, pick **Path 1**.

Level 1 - Must do (about 6 to 8 hours total)

Step 1 - Rebuild from a blank file (2 hours)

1. Close all notes and videos
2. Write this skeleton in comments first:

```
# 1. imports, seed, device
# 2. data (train / val / test)
# 3. model
# 4. loss + optimizer
# 5. train loop + validation loop
# 6. test evaluation
# 7. plots
# 8. save model
```

3. Fill in each part yourself
4. Stuck for 10 minutes? Look up **only one function** in the docs
5. Keep a list called “Things I had to look up”. This is honest and valuable

Step 2 - Get results (30 minutes)

- ☐ Final test accuracy
- ☐ Loss curve image saved to results/
- ☐ Confusion matrix image saved to results/

Step 3 - Explain it in 6 short parts (1 hour)

Write 3 to 5 simple sentences for each. Use the sentence starters:

#	Part	Starter
1	Problem	“This project teaches a computer to ____ because ____.”
2	Input / Output	“The input is ____ with shape _____. The output is ____.”
3	How the model works	“The data goes through ____, then ____, then ____.”

#	Part	Starter
4	Loss function Training	"I used ____ loss because it ____."
5		"I trained with ____ optimizer, learning rate ____, batch size ____, for ____ epochs."
6	Evaluation	"The model got ____ accuracy. It struggled with _____. Next time I would _____."

Step 4 - Tidy the code (45 minutes)

- ☐ Meaningful names (train_loader, not dl1)
- ☐ Short comment or docstring on each function
- ☐ Seeds set
- ☐ Restart and Run All works
- ☐ requirements.txt created (list: torch, numpy, matplotlib)
- ☐ .gitignore includes data/, *.pt, __pycache__/

Step 5 - README (45 minutes)

Copy this template and fill it in:

```
# <Project name>
One sentence: what it does.

## Problem
## Dataset
## Model
## Training setup
## Results (table + 2 images)
## What I learned
## Things I had to look up
## How to run
```

Step 6 - Publish (45 minutes)

- ☐ Push to GitHub
- ☐ Update the root README with a table of all 10 days (Day, Topic, What I built, Key result)
- ☐ Post a short LinkedIn note (template below)

Beginner LinkedIn post (copy and edit)

I just finished a 10-day ML and Deep Learning challenge, from NumPy to Transformers. I built: linear regression from scratch, a neural network in NumPy, a PyTorch CNN that hits ____% on MNIST, and self-attention. My top 3 lessons: 1. Accuracy can be misleading on rare events like fraud. 2. Backprop is just the chain rule going backwards. 3. Attention is a weighted average of words. Code: . Next up: embeddings and RAG. #MachineLearning #DeepLearning #PyTorch #LearningInPublic

Hints

- *Can't start?* Type the 8 skeleton comments first. Then fill in only #1 and #2. Run. Then #3. Small wins add up.
- *Model won't train?* Check, in order: labels' dtype and shape, learning rate, did you call zero_grad()?
- *Can't explain a part?* Draw it: boxes with shapes written on the arrows.
- *README feels long?* Fill the template with one or two sentences per heading. Done is better than perfect.

- *Lookup log:* Every time you open the docs, add one line. It is proof you did the work yourself.

You are done when...

- ☐ Code runs from top to bottom
- ☐ README has results and the 6-part explanation
- ☐ Repo is on GitHub
- ☐ You told someone about it (post, friend or blog)

If you get stuck

Problem	Fix
Don't know where to start	Copy the skeleton comments and fill one at a time
Overwhelmed	Do Step 1 and 2 only today; write later
Accuracy low	Check normalization, learning rate, and <code>model.eval()</code>

Level 2 - Expert Challenges (optional)

1. **Feynman test:** explain the model aloud for 5 minutes to a friend with no ML background. Record it. Wherever you hesitate, study again.
2. **Project structure:** split into `data.py`, `model.py`, `train.py`, `evaluate.py`.
3. **Command-line options:** use `argparse` so `python train.py --lr 0.001 --epochs 10` works.
4. **Experiment table:** try 3 learning rates or 2 architectures and compare.
5. **Error analysis:** view mistakes and explain why the model failed on them.
6. **Blog post:** write 800 to 1,200 words (Problem, Intuition, Code, Results, Mistakes, Next steps).
7. **Deploy a demo:** build a small Gradio or FastAPI app where anyone can upload a digit.
8. **Reproducibility:** add a Makefile or a one-line run script and a results seed table (mean +/- std over 3 seeds).

Progress Tracker

Tick each box when done.

Day	Level 1 done	Level 2 tried	README written	Pushed to GitHub
1 NumPy + Pandas	☐	☐	☐	☐
2 Regression	☐	☐	☐	☐
3 Overfitting	☐	☐	☐	☐
4 Metrics	☐	☐	☐	☐
5 Churn project	☐	☐	☐	☐
6 NN from scratch	☐	☐	☐	☐
7 PyTorch	☐	☐	☐	☐
8 MNIST	☐	☐	☐	☐
9 Attention	☐	☐	☐	☐
10 Capstone	☐	☐	☐	☐

Suggested time per day

Day	Level 1	Level 2 (optional)
1	2-3 h	+1-2 h
2	2-3 h	+2 h
3	3 h	+2 h
4	3 h	+2 h
5	4-5 h	+3 h
6	4 h	+3 h
7	3 h	+2 h
8	4 h	+3 h
9	3 h	+3 h
10	6-8 h	+4 h

Top 8 beginner mistakes

Mistake	Quick fix
Copy-pasting without understanding	Retype it and explain each line
Scaling before splitting	Split first, or use a Pipeline
Judging only by accuracy	Also check precision, recall, confusion matrix
Using the test set to tune	Tune with validation or CV; test once at the end
Forgetting <code>zero_grad()</code>	Always use the 5-line step
Forgetting <code>model.eval()</code>	Use it, with <code>torch.no_grad()</code> , when validating
Wrong label shape	Print <code>.shape</code> of everything
Plots without labels	Add title, axis names, legend

Remember: finished and simple beats perfect and unfinished. Do Level 1 every day, and add Level 2 challenges as you grow.

Solutions Appendix

Stop. Have you tried Level 1 and the hints? If yes, go ahead. Read one solution, understand each line, **close it**, and type your own version from memory.

Notes on the solutions:

- Code marked as “tested” was run and produced the numbers shown (on stand-in data where a download was needed).
- PyTorch solutions (Days 7 to 9) follow your Day notes but were not run while building this guide. Check shapes first if something errors.
- Your exact numbers will differ slightly from the sample numbers. That is normal.

Solution - Day 1

Assignment A: Cosine similarity (tested)

```
import numpy as np

def cosine(a, b):
    """Cosine similarity of two 1-D vectors."""
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

print(round(cosine(np.array([1, 0]), np.array([1, 1])), 3)) # 0.707

def cosine_matrix(A, B, eps=1e-12):
    """A: (n, d), B: (m, d) -> (n, m) similarity table, no loops."""
    A_n = A / (np.linalg.norm(A, axis=1, keepdims=True) + eps)
    B_n = B / (np.linalg.norm(B, axis=1, keepdims=True) + eps)
    return A_n @ B_n.T

np.random.seed(0)
A = np.random.rand(4, 8)
B = np.random.rand(5, 8)
S = cosine_matrix(A, B)
print(S.shape) # (4, 5)

# Prove it matches the slow loop version
S_slow = np.zeros((4, 5))
for i in range(4):
    for j in range(5):
        S_slow[i, j] = cosine(A[i], B[j])
print(np.allclose(S, S_slow)) # True
print(np.allclose(np.diag(cosine_matrix(A, A)), 1)) # True
```

Why it works: dividing each row by its length makes every vector length 1, so the “bottom” of the formula is 1 and only the dot product remains. `A_n @ B_n.T` computes every dot product in one shot. The `eps` stops division by zero when a row is all zeros.

Assignment B: Titanic cleaning (tested on a stand-in table)

```
import pandas as pd

df = pd.read_csv("titanic.csv")
print(df.shape); print(df.isna().sum())

df = df.drop(columns=["Cabin", "Name", "Ticket", "PassengerId"])
df["Age"] = df["Age"].fillna(df["Age"].median())
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
df = df.drop_duplicates()
df["Sex"] = df["Sex"].map({"male": 0, "female": 1})
df = pd.get_dummies(df, columns=["Embarked"], drop_first=True, dtype=int)

assert df.isna().sum().sum() == 0 # safety net
df.to_csv("titanic_clean.csv", index=False)

print(df.groupby("Sex")["Survived"].mean()) # women about 0.74, men about 0.19
print(df.groupby("Pclass")["Survived"].mean()) # 1st about 0.63, 2nd 0.47, 3rd 0.24
```

Why these choices: Cabin is about 77% empty; Name, Ticket, PassengerId are unique per person, so they cannot help predict. Median is used for Age because a few very old passengers would pull the mean up. `dtype=int` makes the dummy columns 0/1 numbers instead of True/False.

Sample README answers

I cleaned the Titanic data by dropping **4** columns and filling missing **Age and Embarked** with the **median and the most common port**. Women survived at about **74%** and men

at about **19%**. First class passengers survived **more often** than third class (about 63% vs 24%). My cosine similarity matrix has shape **(4, 5)**.

Level 2 starter solutions

```
# Speed race (Expert 1)
import time
A = np.random.rand(500, 64); B = np.random.rand(500, 64)
t = time.time(); S_fast = cosine_matrix(A, B); fast = time.time() - t
t = time.time()
S_slow = np.array([[cosine(a, b) for b in B] for a in A]); slow = time.time() - t
print(f"loop {slow:.3f}s  vectorized {fast:.5f}s  -> {slow/fast:.0f}x faster")

# Heatmap (Expert 3)
import matplotlib.pyplot as plt
plt.imshow(S, cmap="viridis"); plt.colorbar(label="cosine similarity")
plt.title("Cosine similarity: A rows vs B rows")
plt.xlabel("B row"); plt.ylabel("A row"); plt.show()
```

Solution - Day 2

Linear regression from scratch (tested)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

np.random.seed(42)
X = 2 * np.random.rand(100, 1)
y = 4 + 3 * X + np.random.randn(100, 1)

def fit(X, y, lr=0.1, epochs=200):
    n = len(X)
    w, b = 0.0, 0.0
    losses = []
    for _ in range(epochs):
        y_pred = w * X + b                # 1. predict
        error = y_pred - y                # 2. how wrong
        losses.append(np.mean(error ** 2)) # 3. loss (MSE)
        dw = (2 / n) * np.sum(error * X) # 4. gradients
        db = (2 / n) * np.sum(error)
        w -= lr * dw                      # 5. step downhill
        b -= lr * db
    return w, b, losses

w, b, losses = fit(X, y)
sk = LinearRegression().fit(X, y)
print("scratch:", round(w, 3), round(b, 3))                # 2.771 4.214
print("sklearn:", round(sk.coef_[0][0], 3), round(sk.intercept_[0], 3)) # 2.77 4.215

plt.plot(losses); plt.title("Loss curve"); plt.xlabel("epoch"); plt.ylabel("MSE"); plt.show()
plt.scatter(X, y, s=10); xs = np.array([[0], [2]])
plt.plot(xs, w * xs + b, "r", label="my line"); plt.legend(); plt.show()
```

What you should see: loss starts near 50 and falls to about 0.8. Your line and scikit-learn's match to about 2 decimals. Both are near (2.8, 4.2), not exactly (3, 4), because random noise nudges the best fit.

Sample README answers

My gradient descent found $w = 2.77$ and $b = 4.21$ (true values 3 and 4). scikit-learn found $w = 2.77$ and $b = 4.215$. The loss curve **falls quickly and then flattens**. When I made the learning rate too large, **the loss exploded to huge numbers instead of shrinking**.

Level 2: learning-rate lab (tested)

```
for lr in [0.001, 0.01, 0.1, 0.5, 1.5]:
    _, _, l = fit(X, y, lr=lr)
    plt.plot(l, label=f"lr={lr}")
plt.yscale("log"); plt.legend(); plt.title("Learning-rate lab"); plt.show()
```

Final losses were about 10.5 (0.001, too slow), 0.88 (0.01, ok), 0.81 (0.1, good), and astronomically large for 0.5 and 1.5 (explodes).

Level 2: mini-batch update

```
def fit_minibatch(X, y, lr=0.05, epochs=100, batch=16):
    n = len(X); w = b = 0.0; losses = []
    for _ in range(epochs):
        idx = np.random.permutation(n)
        for s in range(0, n, batch):
```

```
    i = idx[s:s + batch]
    err = w * X[i] + b - y[i]
    w -= lr * (2 / len(i)) * np.sum(err * X[i])
    b -= lr * (2 / len(i)) * np.sum(err)
    losses.append(np.mean((w * X + b - y) ** 2))
return w, b, losses
```

Solution - Day 3

Overfitting, Ridge/Lasso, cross-validation (tested)

```
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt

np.random.seed(0)
X = np.sort(np.random.rand(60, 1) * 6 - 3, axis=0)
y = (0.5 * X**3 - X**2 + 2 * X + np.random.randn(60, 1) * 3).ravel()
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.3, random_state=1)

train_err, val_err = [], []
for d in range(1, 16):
    m = make_pipeline(PolynomialFeatures(d), StandardScaler(), LinearRegression())
    m.fit(X_train, y_train)
    train_err.append(mean_squared_error(y_train, m.predict(X_train)))
    val_err.append(mean_squared_error(y_val, m.predict(X_val)))
best = int(np.argmin(val_err)) + 1
print("best degree:", best)                                # 3

plt.plot(range(1, 16), train_err, "o-", label="train")
plt.plot(range(1, 16), val_err, "s-", label="validation")
plt.axvline(best, ls="--", c="gray"); plt.yscale("log")
plt.xlabel("Polynomial degree"); plt.ylabel("MSE"); plt.legend(); plt.show()

for name, reg in [("Linear", LinearRegression()), ("Ridge", Ridge(alpha=1)),
                  ("Lasso", Lasso(alpha=0.1, max_iter=50000))]:
    m = make_pipeline(PolynomialFeatures(15), StandardScaler(), reg).fit(X_train, y_train)
    print(name, round(mean_squared_error(y_val, m.predict(X_val)), 2))

models = {
    "Deg3 Linear": make_pipeline(PolynomialFeatures(3), StandardScaler(), LinearRegression()),
    "Deg15 Linear": make_pipeline(PolynomialFeatures(15), StandardScaler(), LinearRegression()),
    "Deg15 Ridge": make_pipeline(PolynomialFeatures(15), StandardScaler(), Ridge(alpha=1)),
}
for name, m in models.items():
    s = -cross_val_score(m, X, y, cv=5, scoring="neg_mean_squared_error")
    print(name, round(s.mean(), 2), round(s.std(), 2))
```

Sample output: best degree 3. Validation MSE at degree 15: Linear about 15, Ridge about 8.9, Lasso about 8.7. CV: Deg3 about 16 (std 11), Deg15 Linear **astronomically large** (about 3e11), Deg15 Ridge about 8.5 (std 5).

Reading it: plain degree-15 Linear is wildly unstable on some folds (overfitting at its worst). Ridge and Lasso tame it. The big std on Deg3 tells you a score from one split can mislead, which is why we use CV.

Sample README answers

Training error kept going **down** as degree increased, but validation error went down until degree **3** and then went **up**. This shows **overfitting**. At degree 15, **Ridge/Lasso** performed **better** than plain Linear because regularization **penalizes huge weights and keeps the curve smooth**. The best model in cross-validation was **Deg15 Ridge** (or **Deg3 Linear** on a different seed) with mean error about **8.5**.

Level 2: alpha experiment (tested)

```
for a in (0.01, 1, 100):
    m = make_pipeline(PolynomialFeatures(15), StandardScaler(), Ridge(alpha=a)).fit(X_train, y_train)
    print("alpha", a, round(mean_squared_error(y_val, m.predict(X_val)), 2))
# alpha 0.01 -> 9.96 (still a bit wild), 1 -> 8.92 (good), 100 -> 35.88 (too strong: underfits)
```

Level 2: leakage demo

```
# WRONG: scaler sees ALL rows, including validation data
Xp = PolynomialFeatures(3).fit_transform(X)
X_all = StandardScaler().fit_transform(Xp)
# RIGHT: put the scaler inside a Pipeline so each CV fold learns it from its own train part
```

Solution - Day 4

Fraud detection comparison (tested on stand-in data)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, HistGradientBoostingClassifier
from sklearn.metrics import (confusion_matrix, precision_score, recall_score,
                             f1_score, roc_auc_score)

df = pd.read_csv("creditcard.csv")
print(df["Class"].value_counts(normalize=True))          # about 0.17% fraud

X = df.drop(columns=["Class"]); y = df["Class"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

X_train, X_test = X_train.copy(), X_test.copy()
scaler = StandardScaler().fit(X_train[["Amount", "Time"]]) # learn from TRAIN only
X_train[["Amount", "Time"]] = scaler.transform(X_train[["Amount", "Time"]])
X_test[["Amount", "Time"]] = scaler.transform(X_test[["Amount", "Time"]])

models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, class_weight="balanced"),
    "Random Forest": RandomForestClassifier(n_estimators=100, class_weight="balanced",
                                           n_jobs=-1, random_state=42),
    "Gradient Boosting": HistGradientBoostingClassifier(random_state=42), # fast version
}
rows = []
for name, m in models.items():
    m.fit(X_train, y_train)
    pred = m.predict(X_test)
    prob = m.predict_proba(X_test)[ :, 1] # probabilities for AUC
    print(name); print(confusion_matrix(y_test, pred)) # [[TN, FP], [FN, TP]]
    rows.append({"Model": name,
                 "Precision": precision_score(y_test, pred),
                 "Recall": recall_score(y_test, pred),
                 "F1": f1_score(y_test, pred),
                 "ROC-AUC": roc_auc_score(y_test, prob)})
print(pd.DataFrame(rows).round(3))
```

Typical findings on the real dataset: Logistic Regression has high recall (about 0.9) but very low precision (many false alarms). Random Forest has high precision (about 0.9) with good recall (about 0.8). Boosting is strong overall. (On the small stand-in data used for testing, numbers were very different; ignore them.)

Self-check answer

TP=40, FP=10, FN=20, TN=930: precision = $40/50 = \mathbf{0.80}$, recall = $40/60 = \mathbf{0.667}$, F1 = $\mathbf{0.727}$. Accuracy would be $970/1000 = 97\%$, which hides that 20 of 60 frauds were missed.

Sample README answers

Only **0.17%** of transactions are fraud, so a model that always says “not fraud” gets about **99.83%** accuracy but **0%** recall. My best recall was **(your number)** (model: **Logistic Regression**). My best precision was **(your number)** (model: **Random Forest**). For a bank I would choose **Random Forest or Boosting** because missing fraud costs **real money and trust** while a false alarm only **annoys a customer for a moment**.

Level 2 starters

```
# Dumb baseline
import numpy as np
print("always-no-fraud accuracy:", 1 - y_test.mean(), " recall: 0.0")

# Threshold tuning (tested): lower threshold -> more recall, less precision
prob = models["Random Forest"].predict_proba(X_test)[: , 1]
for th in (0.3, 0.5, 0.7):
    p = prob > th
    print(th, round(precision_score(y_test, p), 2), round(recall_score(y_test, p), 2))

# Precision-recall curves
from sklearn.metrics import precision_recall_curve
import matplotlib.pyplot as plt
for name, m in models.items():
    pr, rc, _ = precision_recall_curve(y_test, m.predict_proba(X_test)[: , 1])
    plt.plot(rc, pr, label=name)
plt.xlabel("Recall"); plt.ylabel("Precision"); plt.legend(); plt.show()
```

Solution - Day 5

Telco churn pipeline (tested on a stand-in table)

```
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score

df = pd.read_csv("WA_Fn-UseC_-Telco-Customer-Churn.csv")
print(df.shape, df["Churn"].value_counts(normalize=True))

df["TotalCharges"] = pd.to_numeric(df["TotalCharges"], errors="coerce")
df["TotalCharges"] = df["TotalCharges"].fillna(df["TotalCharges"].median())
df = df.drop(columns=["customerID"])
df["Churn"] = df["Churn"].map({"Yes": 1, "No": 0})

X = df.drop(columns=["Churn"]); y = df["Churn"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

num_cols = ["tenure", "MonthlyCharges", "TotalCharges"]
cat_cols = [c for c in X.columns if c not in num_cols]

pre = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
])
pipes = {
    "LogReg": Pipeline([("pre", pre),
                        ("clf", LogisticRegression(max_iter=1000, class_weight="balanced"))]),
    "RandomForest": Pipeline([("pre", pre),
                              ("clf", RandomForestClassifier(n_estimators=300,
                                                            class_weight="balanced", random_state=42))]),
}

cv = StratifiedKFold(5, shuffle=True, random_state=42)
for name, p in pipes.items():
    r = cross_validate(p, X_train, y_train, cv=cv, scoring=["recall", "roc_auc"])
    print(name, {k: round(v.mean(), 3) for k, v in r.items() if k.startswith("test_")})

best = pipes["LogReg"].fit(X_train, y_train)  # chosen from CV results
pred = best.predict(X_test); prob = best.predict_proba(X_test)[:, 1]
print(classification_report(y_test, pred))
print(confusion_matrix(y_test, pred)); print("ROC-AUC", roc_auc_score(y_test, prob))

# Top features (Random Forest)
rf = pipes["RandomForest"].fit(X_train, y_train)
names = rf.named_steps["pre"].get_feature_names_out()
imp = pd.Series(rf.named_steps["clf"].feature_importances_, index=names)
imp.sort_values().tail(10).plot.barh(title="Top 10 features")
```

Real-data expectation: ROC-AUC about 0.84, churn recall about 0.75 with balanced weights. Typical drivers: short tenure, month-to-month contract, fiber optic internet, high monthly charges, no tech support.

Sample README

Problem: A telecom wants to predict which customers will leave so it can offer them deals.
Data: 7,043 customers, about 27% churn. **Method:** Pipeline with scaling and one-hot encoding, models: **Logistic Regression and Random Forest**, checked with 5-fold CV.
Results: best model **Logistic Regression** with ROC-AUC about **0.84** and recall about

0.75 on the test set. **Insights:** customers with **month-to-month** contracts, **short** tenure and **fiber optic** internet are most likely to leave. **Recommendation: offer discounts for one- or two-year contracts to new month-to-month customers.**

Level 2 starters

```
# Grid search
from sklearn.model_selection import GridSearchCV
grid = GridSearchCV(pipes["RandomForest"],
                    {"clf__max_depth": [5, 10, None], "clf__n_estimators": [100, 300]},
                    cv=5, scoring="roc_auc").fit(X_train, y_train)
print(grid.best_params_, round(grid.best_score_, 3))

# Save and reuse
import joblib
joblib.dump(best, "churn.pkl")
model = joblib.load("churn.pkl")
def predict_one(customer_dict):
    return model.predict_proba(pd.DataFrame([customer_dict]))[0, 1]
```

Solution - Day 6

2-layer network in NumPy (tested)

```
import numpy as np
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

np.random.seed(42)
X, y = make_moons(1000, noise=0.2, random_state=42); y = y.reshape(-1, 1)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

def relu(z):    return np.maximum(0, z)
def sigmoid(z): return 1 / (1 + np.exp(-z))
def bce(y, p, eps=1e-9):
    p = np.clip(p, eps, 1 - eps)
    return -np.mean(y * np.log(p) + (1 - y) * np.log(1 - p))

n_in, n_h = 2, 16
W1 = np.random.randn(n_in, n_h) * np.sqrt(2 / n_in); b1 = np.zeros((1, n_h))
W2 = np.random.randn(n_h, 1) * np.sqrt(2 / n_h);   b2 = np.zeros((1, 1))

def forward(X):
    z1 = X @ W1 + b1; a1 = relu(z1)
    z2 = a1 @ W2 + b2; a2 = sigmoid(z2)
    return z1, a1, z2, a2

lr, N, losses = 0.1, len(X_train), []
for ep in range(2000):
    z1, a1, z2, a2 = forward(X_train)
    losses.append(bce(y_train, a2))

    dz2 = (a2 - y_train) / N                # backward: start at the end
    dW2 = a1.T @ dz2; db2 = dz2.sum(0, keepdims=True)
    da1 = dz2 @ W2.T; dz1 = da1 * (z1 > 0)  # ReLU passes gradient only where z1 > 0
    dW1 = X_train.T @ dz1; db1 = dz1.sum(0, keepdims=True)

    if ep == 0:                             # gradient check on one weight
        eps = 1e-5; old = W1[0, 0]
        W1[0, 0] = old + eps; lp = bce(y_train, forward(X_train)[3])
        W1[0, 0] = old - eps; lm = bce(y_train, forward(X_train)[3]); W1[0, 0] = old
        print("analytic", dW1[0, 0], "numeric", (lp - lm) / (2 * eps))

    W1 -= lr * dW1; b1 -= lr * db1; W2 -= lr * dW2; b2 -= lr * db2

acc = lambda X, y: np.mean((forward(X)[3] > 0.5) == y)
print("loss", round(losses[0], 3), "->", round(losses[-1], 3))
print("train acc", acc(X_train, y_train), "test acc", acc(X_test, y_test))
```

Sample output: analytic and numeric gradients agree to about 9 decimals (0.001998704977 vs 0.001998704979). Loss falls from about 1.17 to 0.12. Train accuracy about 96%, test about 97%.

Sample README answers

My network has **2** inputs, **16** hidden neurons and **1** output. It reached about **97%** test accuracy. The loss started at about **1.2** and ended at about **0.12**. Backpropagation means **using the chain rule to work out how much each weight contributed to the loss, going backwards from the output**. A network without a hidden layer only reaches about **85%** because its boundary can only be a **straight line**.

Level 2 starters

```
# Decision boundary
import matplotlib.pyplot as plt
```

```
xx, yy = np.meshgrid(np.linspace(-2, 3, 200), np.linspace(-1.5, 2, 200))
grid = np.c_[xx.ravel(), yy.ravel()]
zz = forward(grid)[3].reshape(xx.shape)
plt.contourf(xx, yy, zz, levels=[0, 0.5, 1], alpha=0.3)
plt.scatter(X_train[:, 0], X_train[:, 1], c=y_train.ravel(), s=8); plt.show()

# Debugging cheat sheet for your journal
# nan loss      -> lr too high, or log(0): use np.clip
# accuracy = 50% -> y has shape (n,) instead of (n,1)
# shape mismatch -> print every .shape; gradient shape == weight shape
```

For the **experiment table**, wrap the training code in a function `train(n_h, lr, activation)` that returns test accuracy, then loop over the values.

Solution - Day 7 (PyTorch, not run while building this guide)

```
import torch, torch.nn as nn
from torch.utils.data import TensorDataset, DataLoader
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

torch.manual_seed(42)
X, y = make_moons(2000, noise=0.25, random_state=42)
X_tr, X_va, y_tr, y_va = train_test_split(X, y, test_size=0.2, random_state=42)

to_t = lambda a: torch.tensor(a, dtype=torch.float32)
train_loader = DataLoader(TensorDataset(to_t(X_tr), to_t(y_tr).view(-1, 1)),
                          batch_size=32, shuffle=True)
val_loader = DataLoader(TensorDataset(to_t(X_va), to_t(y_va).view(-1, 1)),
                        batch_size=256)

model = nn.Sequential(nn.Linear(2, 16), nn.ReLU(), nn.Linear(16, 1))
criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)

train_losses, val_losses = [], []
for epoch in range(60):
    model.train(); total = 0
    for xb, yb in train_loader:
        optimizer.zero_grad()           # 1
        loss = criterion(model(xb), yb)  # 2 + 3
        loss.backward()                 # 4
        optimizer.step()                 # 5
        total += loss.item() * len(xb)
    train_losses.append(total / len(train_loader.dataset))

    model.eval(); vl = correct = 0
    with torch.no_grad():
        for xb, yb in val_loader:
            out = model(xb)
            vl += criterion(out, yb).item() * len(xb)
            correct += ((torch.sigmoid(out) > 0.5).float() == yb).sum().item()
    n = len(val_loader.dataset)
    val_losses.append(vl / n)
    if (epoch + 1) % 10 == 0:
        print(f"epoch {epoch+1}: train {train_losses[-1]:.3f} "
              f"val {val_losses[-1]:.3f} acc {correct / n:.3f}")

plt.plot(train_losses, label="train"); plt.plot(val_losses, label="val")
plt.xlabel("epoch"); plt.ylabel("loss"); plt.legend(); plt.show()
```

Expected: validation accuracy about 90 to 93% (the data has noise 0.25, so even a perfect model cannot reach 100%).

Why each line: `zero_grad()` clears old gradients (PyTorch adds them up otherwise). `backward()` computes gradients. `step()` updates weights. `eval()` and `no_grad()` turn off dropout-style behaviour and save memory.

Sample README answers

PyTorch handles **gradient calculation (backprop)** automatically with `loss.backward()`. My model reached about **92%** validation accuracy. `zero_grad()` is needed because **gradients accumulate, so old ones would be added to new ones**. `model.eval()` is used during validation because **layers like dropout must behave normally when we measure performance**. Compared to my NumPy network from Day 6 the result was **similar**, but the code was **much shorter**.

Level 2: optimizer lab

```
def run(opt_name, lr, epochs=60):
    torch.manual_seed(42)
    m = nn.Sequential(nn.Linear(2, 16), nn.ReLU(), nn.Linear(16, 1))
    opt = (torch.optim.SGD(m.parameters(), lr=lr) if opt_name == "SGD"
           else torch.optim.Adam(m.parameters(), lr=lr))
    hist = []
    for _ in range(epochs):
        m.train()
        for xb, yb in train_loader:
            opt.zero_grad(); criterion(m(xb), yb).backward(); opt.step()
        m.eval()
        with torch.no_grad():
            hist.append(sum(criterion(m(xb), yb).item() * len(xb)
                           for xb, yb in val_loader) / len(val_loader.dataset))
    return hist

for o, lrs in [("SGD", [0.001, 0.01, 0.1]), ("Adam", [0.0001, 0.001, 0.01])]:
    for lr in lrs:
        h = run(o, lr); plt.plot(h, label=f"{o} {lr}")
plt.legend(); plt.yscale("log"); plt.title("Validation loss"); plt.show()
```

Early stopping: keep best_val, a counter, and best_state = copy.deepcopy(model.state_dict()); stop when the counter reaches 10, then model.load_state_dict(best_state).

Solution - Day 8 (PyTorch, not run while building this guide)

```

import torch, torch.nn as nn, copy
from torch.utils.data import DataLoader, random_split
from torchvision import datasets, transforms

torch.manual_seed(42)
device = "cuda" if torch.cuda.is_available() else "cpu"

tf = transforms.Compose([transforms.ToTensor(),
                        transforms.Normalize((0.1307,), (0.3081,))])
full = datasets.MNIST("data", train=True, download=True, transform=tf)
test = datasets.MNIST("data", train=False, download=True, transform=tf)
train, val = random_split(full, [55000, 5000],
                        generator=torch.Generator().manual_seed(42))
train_loader = DataLoader(train, batch_size=64, shuffle=True)
val_loader = DataLoader(val, batch_size=256)
test_loader = DataLoader(test, batch_size=256)

mlp = nn.Sequential(
    nn.Flatten(),
    nn.Linear(784, 256), nn.ReLU(), nn.Dropout(0.3),
    nn.Linear(256, 128), nn.ReLU(), nn.Dropout(0.3),
    nn.Linear(128, 10),
)

cnn = nn.Sequential(
    nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2), # -> 16 x 14 x 14
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2), # -> 32 x 7 x 7
    nn.Flatten(), # -> 1568
    nn.Linear(32 * 7 * 7, 128), nn.ReLU(), nn.Dropout(0.3),
    nn.Linear(128, 10),
)

def run_epoch(model, loader, optimizer=None):
    training = optimizer is not None
    model.train() if training else model.eval()
    total_loss = correct = n = 0
    with torch.set_grad_enabled(training):
        for xb, yb in loader:
            xb, yb = xb.to(device), yb.to(device)
            out = model(xb)
            loss = criterion(out, yb)
            if training:
                optimizer.zero_grad(); loss.backward(); optimizer.step()
            total_loss += loss.item() * len(xb)
            correct += (out.argmax(1) == yb).sum().item(); n += len(xb)
    return total_loss / n, correct / n

criterion = nn.CrossEntropyLoss()

def fit(model, epochs=20, patience=3):
    model.to(device)
    opt = torch.optim.Adam(model.parameters(), lr=1e-3)
    best_val, best_state, bad = 1e9, None, 0
    history = {"train": [], "val": []}
    for ep in range(epochs):
        tl, _ = run_epoch(model, train_loader, opt)
        vl, va = run_epoch(model, val_loader)
        history["train"].append(tl); history["val"].append(vl)
        print(f"epoch {ep+1}: train {tl:.3f} val {vl:.3f} val acc {va:.4f}")
        if vl < best_val:
            best_val, best_state, bad = vl, copy.deepcopy(model.state_dict()), 0
        else:
            bad += 1
            if bad >= patience: break # early stopping
    model.load_state_dict(best_state) # restore best weights
    return history

for name, model in [("MLP", mlp), ("CNN", cnn)]:

```

```
fit(model)
_, test_acc = run_epoch(model, test_loader)    # test ONCE, at the very end
params = sum(p.numel() for p in model.parameters())
print(name, "params", params, "test acc", round(test_acc, 4))
```

Parameter counts: MLP is 235,146 and CNN is about 207,000. Expect MLP about 97 to 98% and CNN about 98.5 to 99%.

Why the CNN wins: it looks at small local patches and reuses the same filter everywhere (shared weights), so it understands shapes and needs fewer parameters.

Sample README answers

The CNN performed **better** because it keeps the **spatial (2D) structure** of the image. Dropout helps by **randomly switching neurons off so the network cannot rely on any single one**. The digits most often confused were **4 and 9** and **3 and 5**.

Level 2 starters

```
# Confusion matrix
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
cnn.eval(); preds, labels = [], []
with torch.no_grad():
    for xb, yb in test_loader:
        preds += cnn(xb.to(device)).argmax(1).cpu().tolist(); labels += yb.tolist()
plt.imshow(confusion_matrix(labels, preds), cmap="Blues"); plt.colorbar()
plt.xlabel("predicted"); plt.ylabel("true"); plt.show()

# 8 mistakes
xb, yb = next(iter(test_loader)); xb, yb = xb.to(device), yb.to(device)
pred = cnn(xb).argmax(1)
wrong = (pred != yb).nonzero().flatten()[:8]
for i, idx in enumerate(wrong):
    plt.subplot(2, 4, i + 1); plt.imshow(xb[idx].cpu().squeeze(), cmap="gray")
    plt.title(f"true {yb[idx].item()} pred {pred[idx].item()}"); plt.axis("off")
plt.show()
```

Solution - Day 9 (PyTorch, not run while building this guide)

```
import math, torch, torch.nn as nn, torch.nn.functional as F
import matplotlib.pyplot as plt

class SelfAttention(nn.Module):
    def __init__(self, d_model, d_k):
        super().__init__()
        self.W_q = nn.Linear(d_model, d_k, bias=False)
        self.W_k = nn.Linear(d_model, d_k, bias=False)
        self.W_v = nn.Linear(d_model, d_k, bias=False)
        self.d_k = d_k

    def forward(self, x, mask=None):
        # x: (B, T, d_model)
        Q, K, V = self.W_q(x), self.W_k(x), self.W_v(x) # (B, T, d_k)
        scores = Q @ K.transpose(-2, -1) / math.sqrt(self.d_k) # (B, T, T)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, float("-inf"))
        weights = F.softmax(scores, dim=-1) # rows sum to 1
        return weights @ V, weights # (B, T, d_k)

torch.manual_seed(0)
x = torch.randn(1, 6, 16)
attn = SelfAttention(16, 16)
out, w = attn(x)
print(out.shape, w.shape) # torch.Size([1, 6, 16]) torch.Size([1, 6, 6])
print(w[0].sum(dim=-1)) # all ones

tokens = ["The", "cat", "sat", "on", "the", "mat"]
plt.imshow(w[0].detach().numpy(), cmap="viridis")
plt.xticks(range(6), tokens, rotation=45); plt.yticks(range(6), tokens)
plt.xlabel("Key (attended T0)"); plt.ylabel("Query (attending)")
plt.colorbar(label="attention weight"); plt.tight_layout(); plt.show()
```

Sample README answers

Attention lets each word **look at every other word and take more information from the ones that matter**. **Q** is what this word is looking for, **K** is what each word offers as a label, **V** is the actual information each word gives. We divide by $\sqrt{d_k}$ because **big dot products push softmax into extreme values where gradients vanish**. Each row of the attention matrix sums to **1** because of **softmax**. Multi-head attention helps because **different heads can focus on different kinds of relationships at once**.

Level 2: causal mask and multi-head

```
# Causal mask: each word sees only itself and earlier words
T = 6
mask = torch.tril(torch.ones(T, T)).unsqueeze(0) # (1, T, T) lower-triangular
out, w = attn(x, mask=mask)
print(w[0]) # everything above the diagonal is exactly 0

# Simple multi-head: run several heads, concatenate, then mix with a final Linear
class MultiHead(nn.Module):
    def __init__(self, d_model, n_heads):
        super().__init__()
        dk = d_model // n_heads
        self.heads = nn.ModuleList([SelfAttention(d_model, dk) for _ in range(n_heads)])
        self.proj = nn.Linear(d_model, d_model)
    def forward(self, x, mask=None):
        return self.proj(torch.cat([h(x, mask)[0] for h in self.heads], dim=-1))
```

Hand check (Practice task)

Weights $[0.20, 0.40, 0.40]$ times values $[1,0], [0,1], [1,1]$: $0.20 \cdot [1,0] + 0.40 \cdot [0,1] + 0.40 \cdot [1,1] = \mathbf{[0.60, 0.80]}$.

Solution - Day 10 (Capstone)

The capstone must be **your own** rebuild, so there is no answer key for the code. Instead, here are the checkpoints and answers.

Self-checks for your own code

Checkpoint	Correct answer
Input shape of one MNIST batch of 64	(64, 1, 28, 28)
Model output shape	(64, 10) (raw scores, one per digit)
Predicted digit	<code>output.argmax(1)</code>
Loss function	<code>CrossEntropyLoss</code> (contains softmax; labels are integer class numbers)
Expected test accuracy	about 98.5 to 99% for a small CNN
Order inside the loop	<code>zero_grad</code> , <code>forward</code> , <code>loss</code> , <code>backward</code> , <code>step</code>

Answers to the self-assessment quiz

1. **100% train accuracy but useless?** The model memorized the training data (overfitting). It fails on new data. Only validation/test scores tell the truth.
2. **Loss curves:** too low a learning rate gives a slow, nearly flat decline; the right rate gives a fast fall that flattens; too high gives a jagged, rising or nan curve.
3. **99.8% accuracy, 0% recall (fraud):** the model says “not fraud” for everything. Only 0.2% of cases are fraud, so accuracy looks great while every fraud is missed.
4. **The 5 lines:** `optimizer.zero_grad()` clears old gradients; `out = model(x)` forward; `loss = criterion(out, y)` measures error; `loss.backward()` computes gradients; `optimizer.step()` updates weights.
5. **Output size** for input 28, kernel 3, padding 1, stride 2: $\text{floor}((28 - 3 + 2) / 2) + 1 = \text{floor}(13.5) + 1 = 14$.
6. **Q, K, V library analogy:** Query = what you’re looking for; Key = the label on each book’s spine; Value = the content inside. You match your query to the labels and read mostly the best-matching books.
7. **BERT vs GPT:** BERT is a bidirectional encoder (sees the whole sentence, good for understanding). GPT is a causal decoder (sees only earlier words, good for generating text).
8. **Why O(T squared)?** Every one of T tokens is scored against every other token, giving a T by T table.

Starter skeleton for the capstone (Path 1: fill in the TODOs yourself)

```
# train.py
import torch, torch.nn as nn

def get_data():
    ... # TODO: datasets, transforms, split 55k/5k/10k, return 3 DataLoaders

class Net(nn.Module):
    def __init__(self):
        super().__init__()
        ... # TODO: conv blocks + linear layers
    def forward(self, x):
        ... # TODO: return logits of shape (B, 10)

def train(model, train_loader, val_loader, epochs):
    ... # TODO: the 5-line loop + validation + early stopping

def evaluate(model, test_loader):
    ... # TODO: accuracy + confusion matrix

if __name__ == "__main__":
```

```
torch.manual_seed(42)
# TODO: wire the pieces together, save plots into results/
```

Level 2 starter: command-line options

```
import argparse
p = argparse.ArgumentParser()
p.add_argument("--lr", type=float, default=1e-3)
p.add_argument("--epochs", type=int, default=10)
args = p.parse_args()
# run with: python train.py --lr 0.001 --epochs 10
```

Filled root README table (example rows)

Day	Topic	What I built	Key result
1	NumPy/Pandas	Vectorized cosine similarity, Titanic cleaning	0 loops, clean CSV
2	Regression	Linear regression from scratch vs sklearn	$w = 2.77$, $b = 4.21$, matches sklearn
3	Overfitting	Degrees 1 to 15, Ridge/Lasso, CV	Best degree = 3
4	Metrics	Fraud detection with 3 models	Recall vs precision trade-off
5	Churn	Pipeline + CV	ROC-AUC about 0.84
6	NN scratch	2-layer NumPy network	97% test accuracy, gradient check passes
7	PyTorch	Training loop, SGD vs Adam	about 92% val accuracy
8	MNIST	MLP vs CNN	CNN about 99%
9	Attention	Self-attention with heatmap	Rows sum to 1
10	Capstone	Clean rebuild	Test accuracy: your number